

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего образования
**«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)**

Институт информационных технологий, математики и механики

Кафедра теоретической, компьютерной и экспериментальной механики

Направление подготовки 01.03.02 Прикладная математика и информатика
Направленность (профиль) программы бакалавриата: Математическое
моделирование и вычислительная математика

ТЕХНИЧЕСКОЕ ОПИСАНИЕ ПРОГРАММЫ
«ParserKFile»

Выполнил:
студент группы 3822Б1ПМмм1
Чернов Кирилл Сергеевич

Руководитель:
доцент кафедры ТКЭМ, к.ф.-м.н.
Дюкина Надежда Сергеевна

г. Нижний Новгород
2025 г.

Оглавление

Название раздела	стр.
1. Назначение и область применения	2
2. Постановка задачи	3
3. Алгоритм работы программы	4
4. Блок-схема алгоритма	6
5. Описание архитектуры программы	9
6. Используемые технологии	12
7. Инструкция по запуску	13
8. Пример работы	14
9. Тестирование	19
10. Приложения	22

1. Назначение и область применения

Программное обеспечение предназначено для автоматической подготовки исходных данных для численного моделирования задач прочности в программном комплексе «ЛОГОС Прочность». Программа обеспечивает обработку К-файла геометрической модели, вычисление начального напряженного состояния грунтового массива под действием силы тяжести и автоматическое формирование CD-файла с наборами ячеек и наборами начальных напряжений, соответствующих требованиям.

Программа применяется при моделировании грунтовых оснований, представленных в виде прямоугольного параллелепипеда, дискретизированного на конечные элементы. В таких моделях напряженное состояние формируется под действием собственного веса материала, и напряжения возрастают с глубиной. Автоматизация вычислений и формирования CD-файла позволяет исключить ошибки, возникающие при ручной подготовке данных, уменьшить трудоемкость процесса и обеспечить корректность дальнейших расчетов в «ЛОГОС Прочность».

Область применения включает учебные, научно-исследовательские и инженерные задачи по подготовке конечно-элементных моделей деформируемых сред, где требуется корректное задание начального поля напряжений. Программа поддерживает структурирование данных в формате *NODE, *ELEMENT_SOLID, CELL_SETS, SET_SOLID и INITIAL_STRESS_SET, что обеспечивает совместимость с расчетным модулем «ЛОГОС Прочность».

2. Постановка задачи

Рассматривается задача определения начального напряженного состояния грунтового основания в виде прямоугольного параллелепипеда. Нижняя грань основания жестко закреплена, а на боковых гранях допускается вертикальное проскальзывание (вертикальная ось принята за Oy). На тело действует только сила тяжести. В результате осадки массива формируется одноосное деформированное состояние.

Напряженное состояние в такой среде определяется системой формул, более подробно разобранных в отчете, в которых используются плотность материала ρ , ускорение свободного падения g и координата y , характеризующая глубину элементарного объема относительно свободной поверхности. Эти зависимости позволяют вычислить неоднородное поле напряжений, линейно растущее с глубиной.

Численное моделирование выполняется методом конечных элементов с использованием явной схемы интегрирования по времени «крест» в программном комплексе «ЛОГОС Прочность». Расчетная модель формируется в модуле «ЛОГОС Препост» и состоит из двух файлов — *.k и *.cd.

К-файл содержит координаты узлов и состав конечных элементов (*NODE, *ELEMENT_SOLID).

CD-файл содержит данные о наборе ячеек, механических свойствах, граничных условиях и начальных напряжениях (CELL_SETS, SET_SOLID, INITIAL_STRESS_SET).

Постановка задачи заключается в автоматическом формировании набора начальных условий для CD-файла на основании анализа структуры К-файла:

- идентификации подобластей и слоев;
- определении высоты слоев и глубин узлов;
- вычислении напряжений по физической модели;
- формировании структур SET_SOLID и INITIAL_STRESS_SET;
- записи корректного CD-файла, пригодного для последующего численного расчета.

Ручное выполнение этих действий является трудоемким, требует высокой аккуратности и подвержено ошибкам, что делает автоматизацию задачи необходимой для эффективной подготовки расчетных моделей.

3. Алгоритм работы программы

1. Выбор входных данных

Пользователь выбирает К-файл, содержащий сеточную модель, сформированную в программе «ЛОГОС», и CD-файл.

2. Чтение структуры К-файла

Приложение выполняет разбор содержимого файла:

- считывает узлы из секции ***NODE**;
- извлекает элементы из секции ***ELEMENT_SOLID**;
- формирует внутренние словари узлов и элементов.

3. Фильтрация элементов по подобласти

Пользователь указывает номер подобласти.

Программа выбирает только те элементы, которые принадлежат заданной подобласти (**element_solid** с нужным SID).

4. Определение структуры грунтового массива

Программа:

- выделяет нижележащий прямоугольный параллелепипед, исключая возможные объекты, расположенные выше;
- определяет фактическую высоту слоя грунта по координатам узлов.

5. Выбор координаты для группировки слоев

Пользователь выбирает ось группировки (X, Y или Z).

Приложение формирует отсортированный набор уникальных координат, определяющих слои модели.

6. Определение слоев и их элементов

Для каждого слоя программа:

- определяет элементы, относящиеся к его диапазону координат;
- формирует список ID элементов (**ELEMENTS**) для последующей записи в **SET_SOLID**.

7. Расчет напряженного состояния

Для каждого найденного элемента вычисляется напряженность согласно физической постановке задачи:

- напряжения определяются по координате глубины;
- результаты заносятся в структуры **INITIAL_STRESS_SET**.

8. Генерация промежуточного файла **_debug.txt**

Программа собирает данные в форматах:

- **CELL_SETS**
- **SET_SOLID**
- **INITIAL_STRESS_SET**

Все данные записываются в промежуточный YAML-файл, затем переводятся в текстовый формат для последующей вставки в CD.

9. Формирование выходного CD-файла

Приложение открывает исходный CD-файл и автоматически вставляет в него

данные из `_debug.txt`.

После сборки структура приводится к корректному формату `.cd`.

10. Получение итогового `_output.cd`

В выходной директории формируется готовый CD-файл, полностью совместимый с программой «ЛОГОС Прочность».

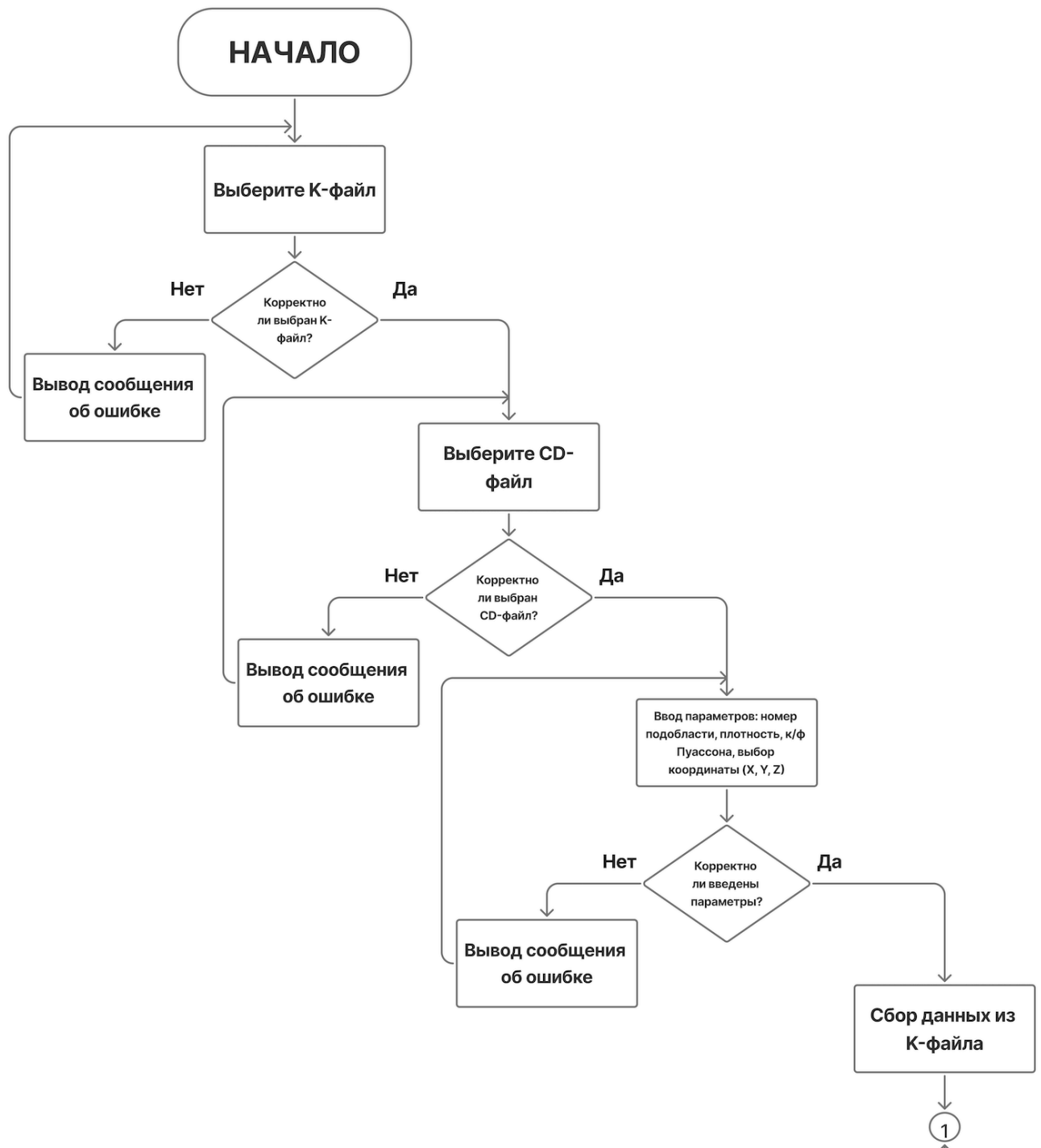
11. Завершение работы

Программа уведомляет пользователя об успешной генерации файла.

В GUI-режиме создается ссылка перехода к папке с результатами.

4. Блок-схемы программы

- Интерфейс





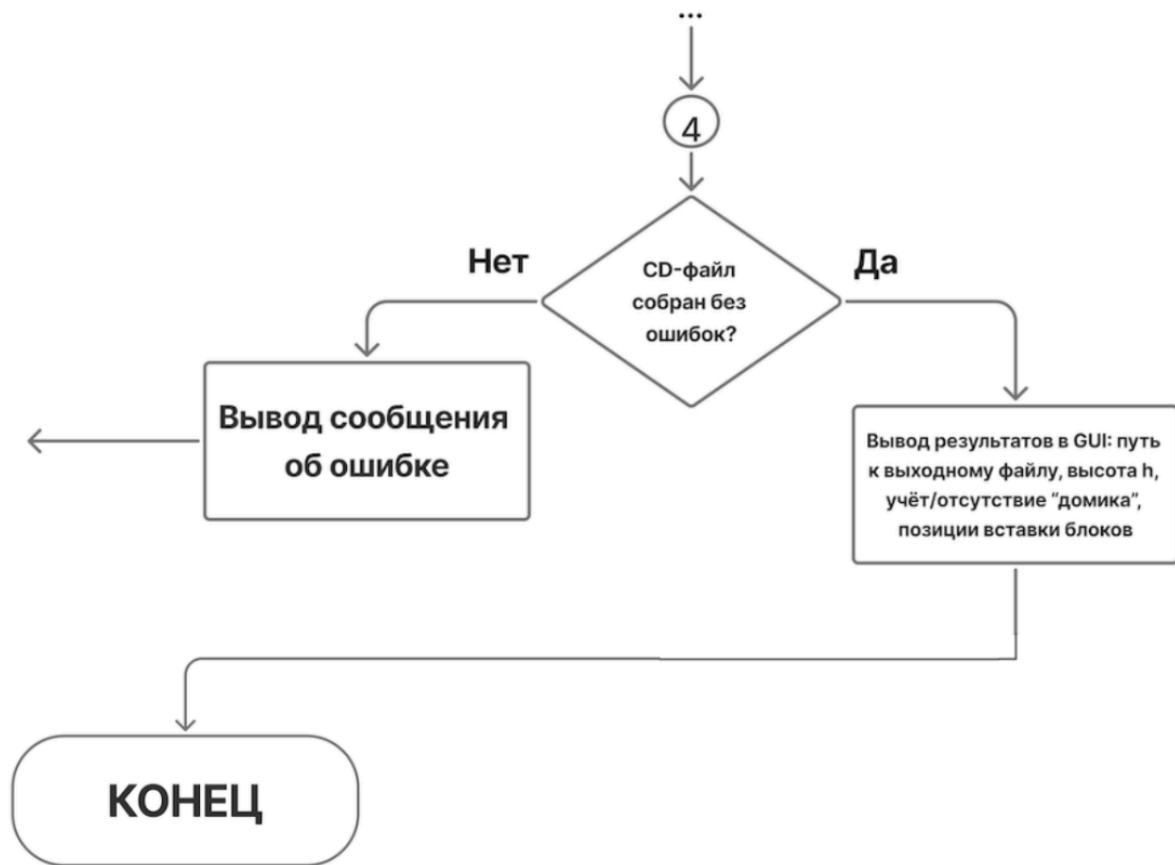


Рисунок 1. Блок-схема UI

- Алгоритм получения данных

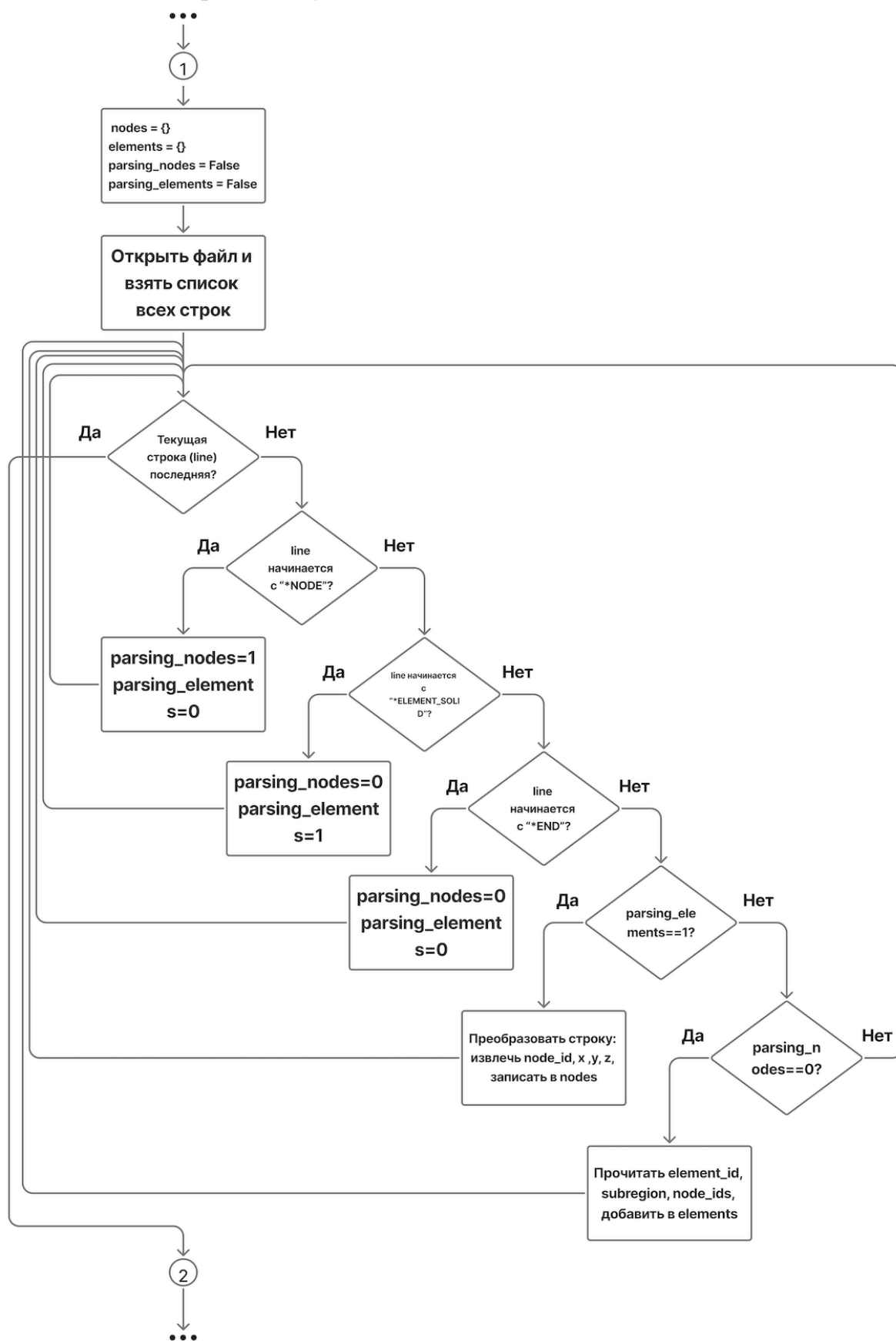
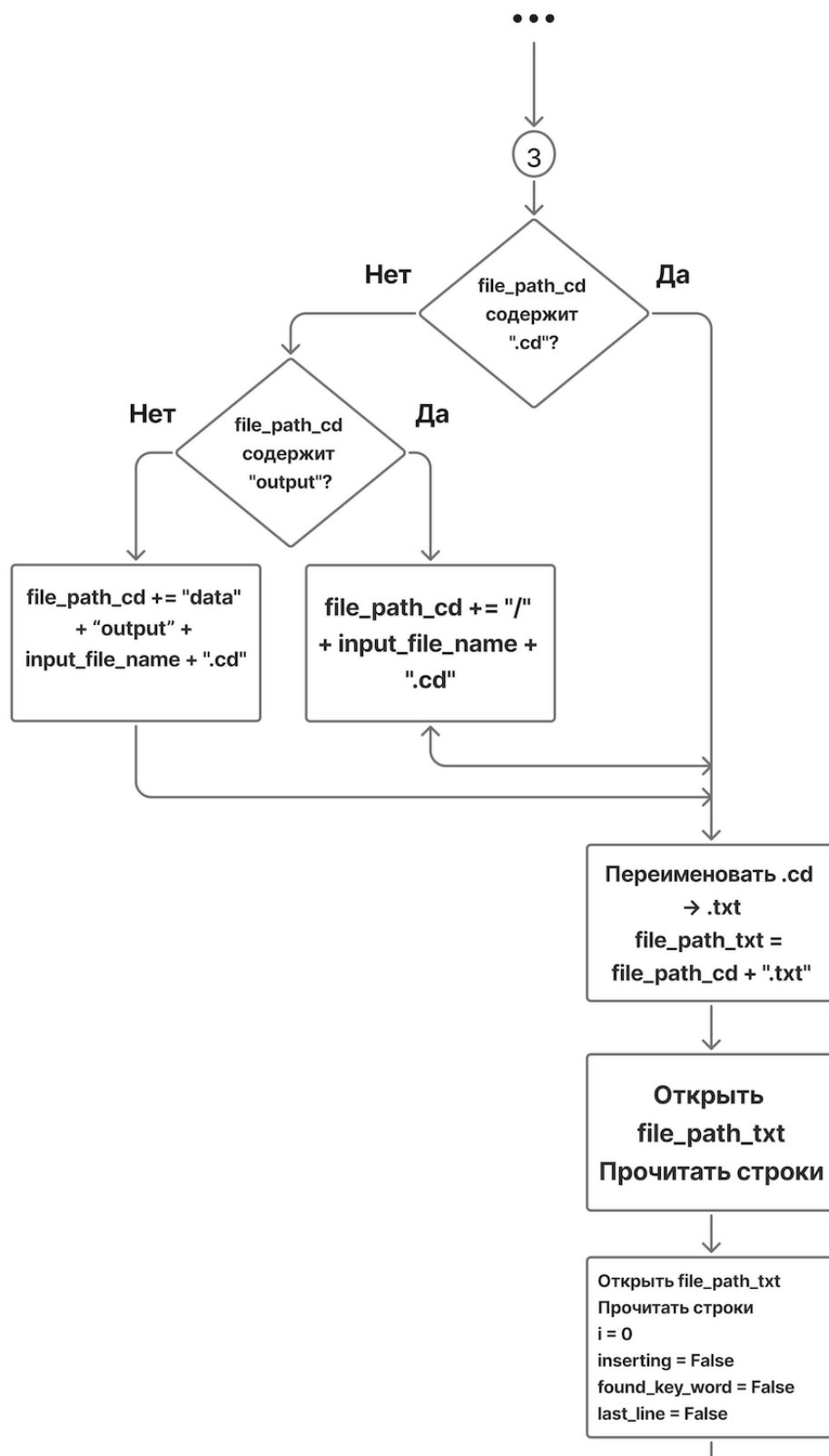
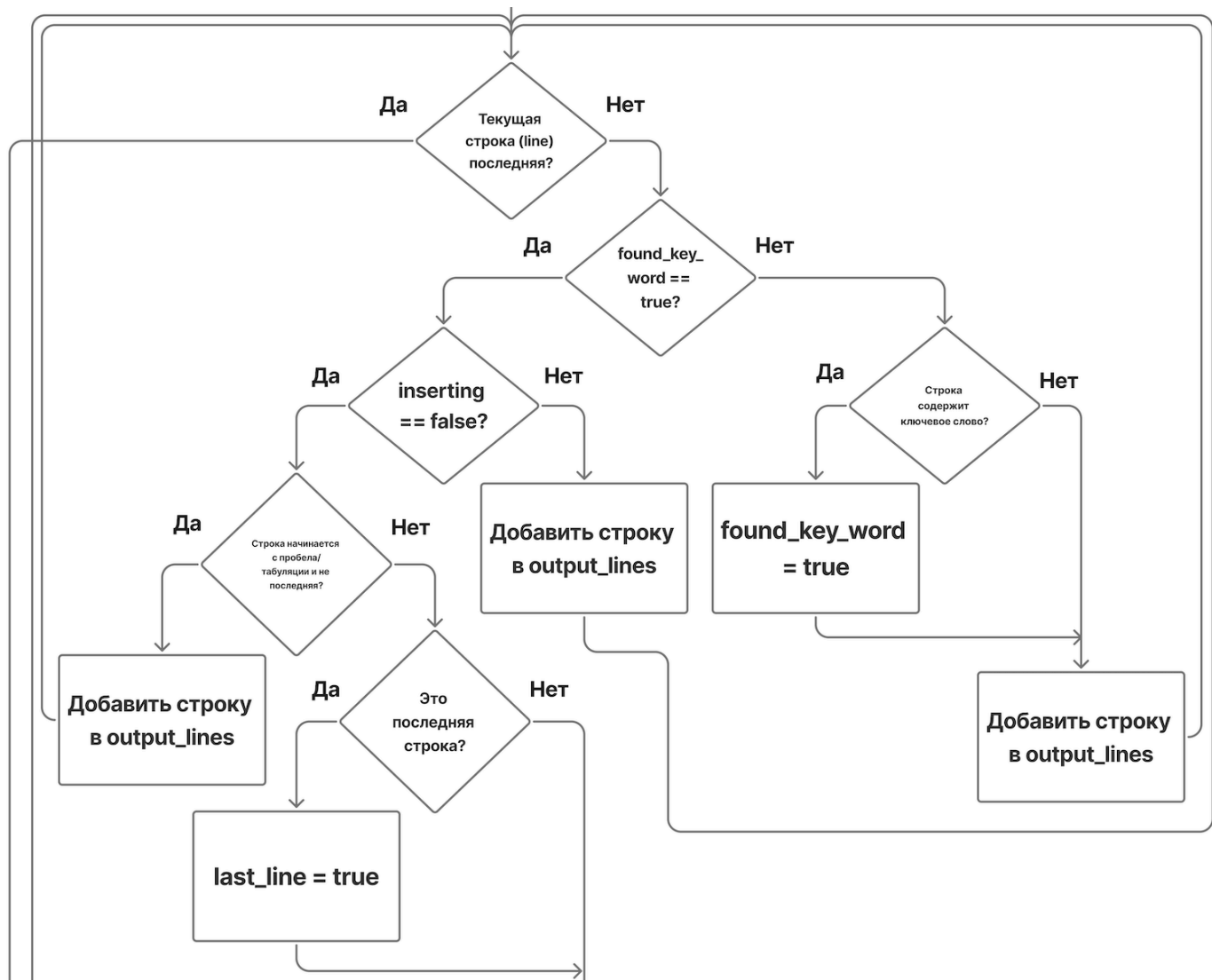


Рисунок 2. Блок-схема алгоритма получения данных

- Алгоритм вставки данных





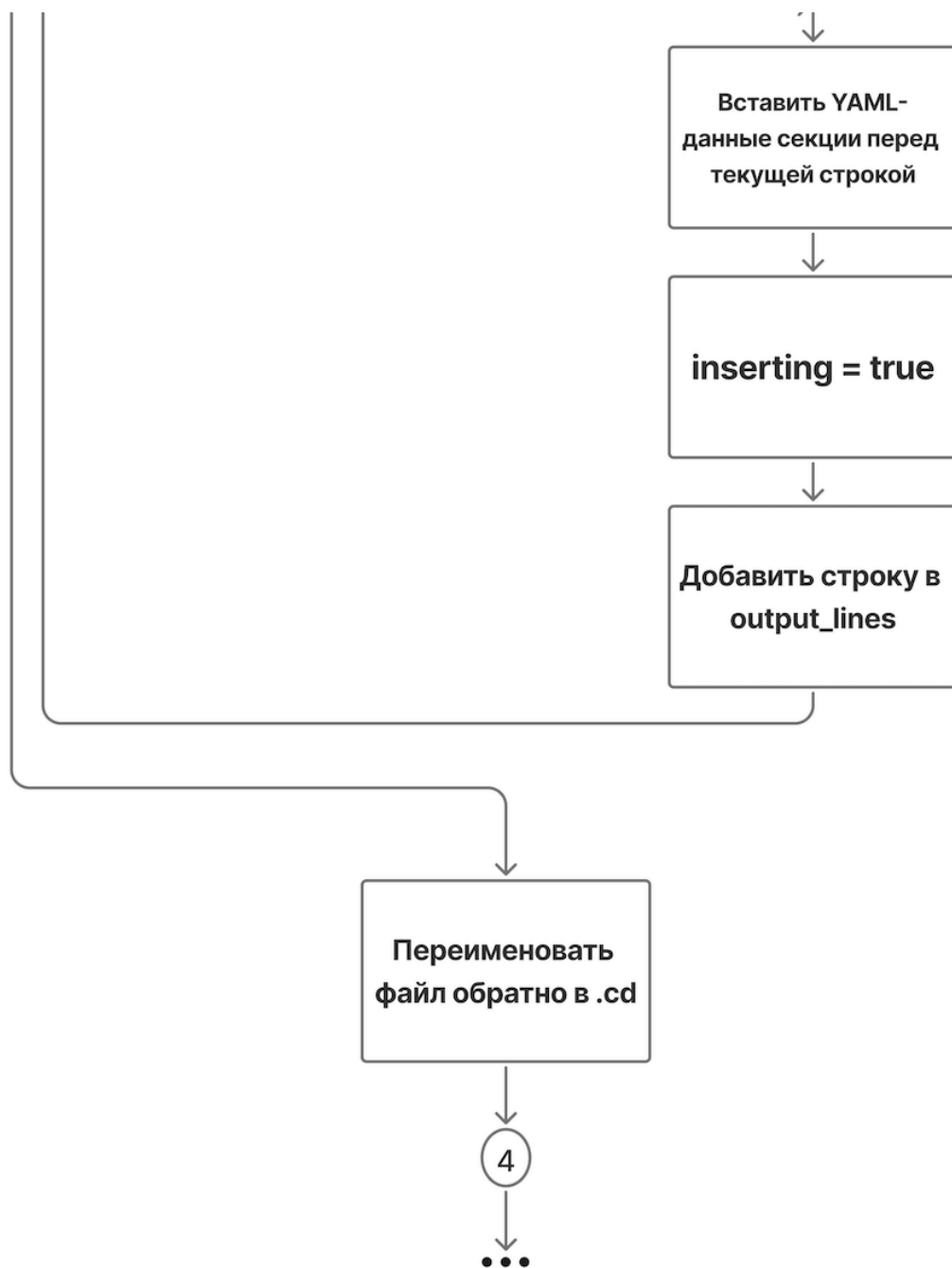


Рисунок 3. Блок-схема алгоритма получения данных

5. Описание архитектуры программы

Архитектура программы представлена в следующем виде (рисунок 4):

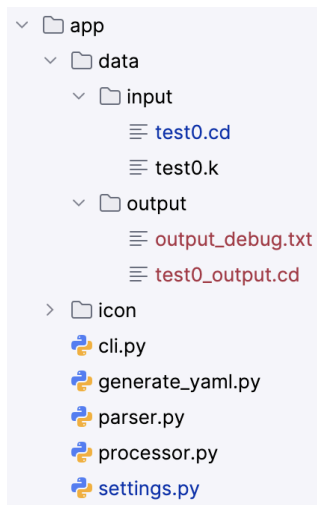


Рисунок 4. Архитектура программы

- data

Назначение:

Каталог для хранения входных и выходных данных.

Содержит:

- **input/** — сюда помещаются К-файлы, содержащие описание узлов, элементов и других параметров геомодели.
- **output/** — в эту папку сохраняются сгенерированные YAML-файлы и CD-файлы, которые используются в дальнейших расчетах.

- icon

Назначение:

Каталог, содержащий иконку программы для сборки **.exe** с визуальным стилем.

Применение:

Используется при упаковке приложения с помощью PyInstaller, чтобы exe-файл имел собственную иконку, например:

```
pyinstaller app.py --icon=icon/app.ico
```

- [cli.py](#)

Назначение:

Файл командной строки для запуска парсера и генерации данных без использования графического интерфейса. Запуск приложения через консоль было сделано для более быстрой и детальной отладки приложения

Функции:

- Принимает аргументы через `click` (например, путь к файлам, координата фильтрации, плотность и т.д.).
 - Обрабатывает данные и генерирует выходные файлы.
 - Позволяет выполнять весь функционал программы с терминала.
-
- `generate_yaml.py`

Назначение:

Генерация выходных файлов в формате YAML и CD.

Ключевые функции:

- `generate_unique_id(...)` - генерация уникального идентификатора, который требуется для `.cd` файла
 - `generate_layer_data(...)` - генерация данных для всех слоев с учетом выбранной координаты
 - `get_output_dir(...)` - получение пути выходных файлов
 - `write_to_yaml(...)` - запись данных в YAML файл
 - `write_to_cd_by_k_word(...)` - запись в `_output.cd` файл из `_debug.txt`
-
- [parser.py](#)

Назначение:

Модуль для парсинга `.k`-файлов.

Ключевые функции:

- `parse_k_file(...)` — читает файл с расширением `.k`, извлекает из него информацию об узлах (`nodes`) и элементах (`elements`).
 - Возвращает словари с координатами и ID элементов, которые затем используются в обработке
-
- [processor.py](#)

Назначение:

Основной модуль для логической обработки данных: фильтрация, группировка, анализ.

Ключевые функции:

- `filter_elements_by_subregion(...)` — выбирает элементы, относящиеся к заданной подобласти.
- `group_nodes_by_coordinate(...)` — группирует узлы по значениям координаты (X, Y или Z).
- `find_elements_for_layer(...)` — находит элементы, принадлежащие определенному слою.
- `find_h_and_home(...)` — определяет границы "домика" и прямоугольной области (используется для оценки высоты слоя).
- settings

Назначение:

Файл конфигурации, содержащий глобальные переменные (глобальная директория для сохранения конечного файла), информацию о флагах, после которых будет происходить вставка данных.

Содержит:

- Название входного файла (без расширения).
- Строки-шаблоны, которые будут использоваться при генерации CD-файлов:
 - `put_cell_sets`
 - `put_stress_set`
 - `put_set_solid`

Преимущества:

Упрощает настройку параметров без необходимости менять код в других файлах.

6. Используемые технологии

Язык реализации:

Приложение разработано на языке Python, что обеспечивает простоту интеграции, широкие возможности для обработки данных и кроссплатформенность.

Библиотеки и инструменты:

- **tkinter** — используется для создания графического интерфейса, обеспечивая удобную визуальную оболочку для работы с K/CD-файлами.
- **click** — применяется для реализации удобного и расширяемого интерфейса командной строки (CLI), позволяя запускать обработку файлов без GUI.
- **PyInstaller** — используется для сборки исполняемых файлов под macOS (.app) и Windows (.exe), обеспечивая переносимость приложения без необходимости установки Python.

7. Инструкция по запуску

1. Клонирование проекта

```
git clone https://github.com/Kurillccc/ParserKFile
```

2. Переходим в корень проекта

```
cd ParserKFile
```

3. Устанавливаем зависимости из requirements.txt

```
python3 -m venv .venv source .venv/bin/activate pip install -r requirements.txt
```

4. Запускаем приложение с графическим интерфейсом

```
python run.py
```

или для работы через консоль (через консоль происходит более детальное описание действий для поиска возможных ошибок)

```
python app/cli.py --input путь/к/файлу.k
```

Также, есть возможность скачивания исполняемого .exe файла

8. Пример работы

Интерфейс

- MacOS (рисунок 5)

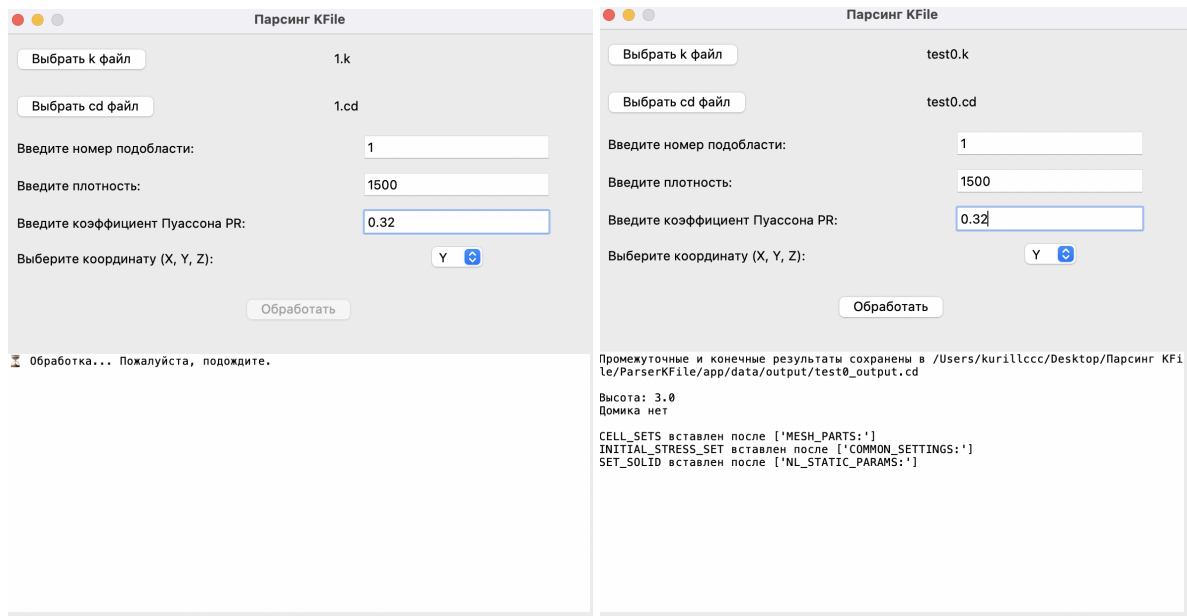


Рисунок 5. Интерфейс программы на macOS

- Windows (рисунок 6)

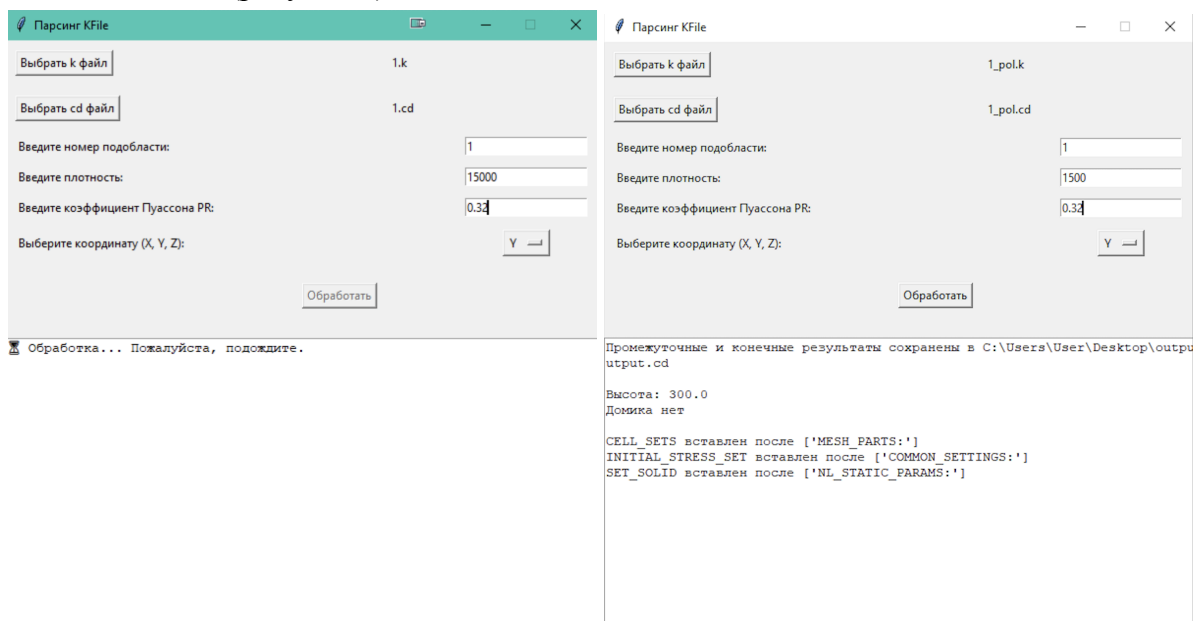


Рисунок 6. Интерфейс программы на windows

Результат обработки:

Как и говорилось ранее, на вход поступают файлы из программы “Логос Прочность”, а на выходе обработанные файлы для дальнейшей работы (рисунок 7)

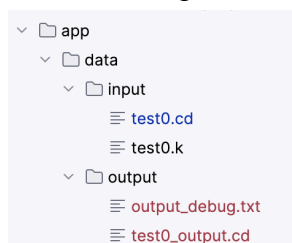


Рисунок 7. Файлы input и output

Файл _debug.txt содержит поля, требуемые для _output.cd (рисунок 8)

CELL_SETS:	INITIAL_STRESS_SET:	SET_SOLID:
- Id: 1 - Name: set1 - Count: 9 _ref_used_: 1 - uid: 6575002a4dce4c89ac4c8e93ce33aa6d - parentUid: '' __excludeRun__: '~' - Id: 2 - Name: set2 - Count: 9 _ref_used_: 1 - uid: 4f2c0fadd3ad468a89489598482bb912 - parentUid: '' __excludeRun__: '~' - Id: 3 - Name: set3 - Count: 9 _ref_used_: 1 - uid: 67fee3ed4b1f4543b8802a5fd7ef0f24 - parentUid: '' __excludeRun__: '~'	- ESID: 1 - SIGXX: -17294.11764705883 - SIGYY: -36750.00000000001 - SIGZZ: -17294.11764705883 - SIGXY: 0 - SIGYZ: 0 - SIGZX: 0 - EPS: 0 - name: set1 - uid: 5fb14d9a87444662abf186b3480a6ea4 - parentUid: '' __excludeRun__: '~' - ESID: 2 - SIGXX: -10376.470588235296 - SIGYY: -22050.000000000004 - SIGZZ: -10376.470588235296 - SIGXY: 0 - SIGYZ: 0 - SIGZX: 0 - EPS: 0 - name: set2 - uid: 87c147d9b11749baa0cfd90837e1e645	- NAME: set1 - SID: 1 - ELEMENTS: - 1 - 2 - 3 - 16 - 18 - 20 - 22 - 24 - 25 - NAME: set2 - SID: 2 - ELEMENTS: - 5 - 7 - 9 - 17 - 19 - 21 - 23 - 26 - 27

Рисунок 8. Поля файла _debug.txt

Далее, полученный файл можно открыть в “Логос Прочность”, и мы увидим обработанную модель, для которой автоматически сформированы наборы ячеек – горизонтальные слои и рассчитано напряжение для каждого слоя.

Обработанная модель показана на рисунке 9

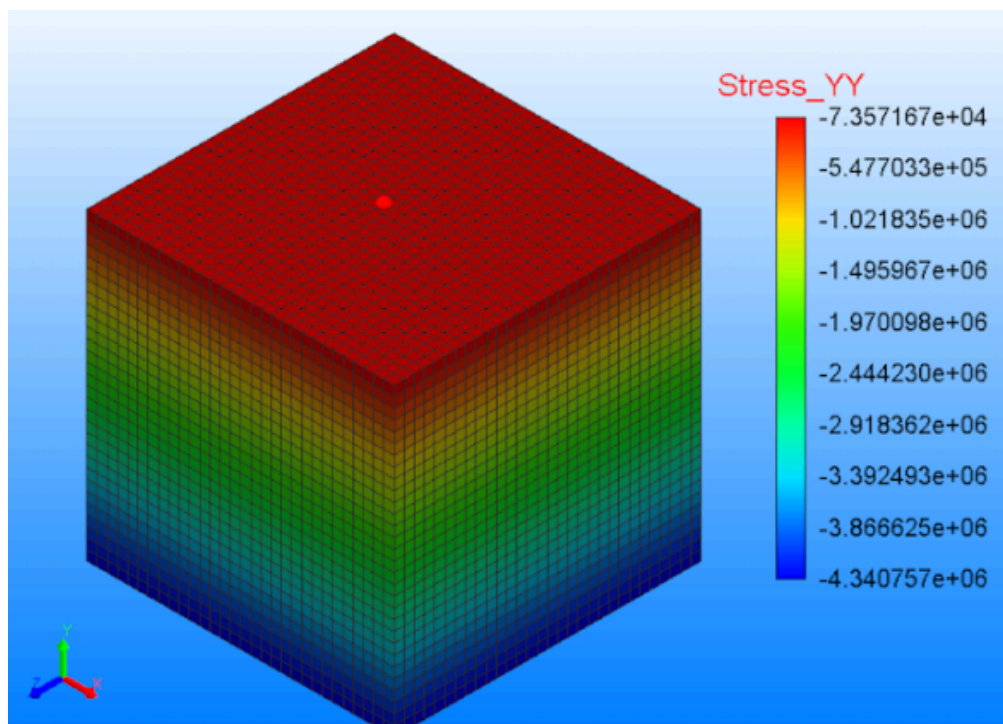


Рисунок 9. Распределение поля напряжений S_{yy}

При расчете осадки массива сплошной среды с применением процедуры динамической релаксации в “Логос Прочность” с необработанным файлом получена временная зависимость для вертикальной скорости точки поверхности массива, показанная на рисунке 10

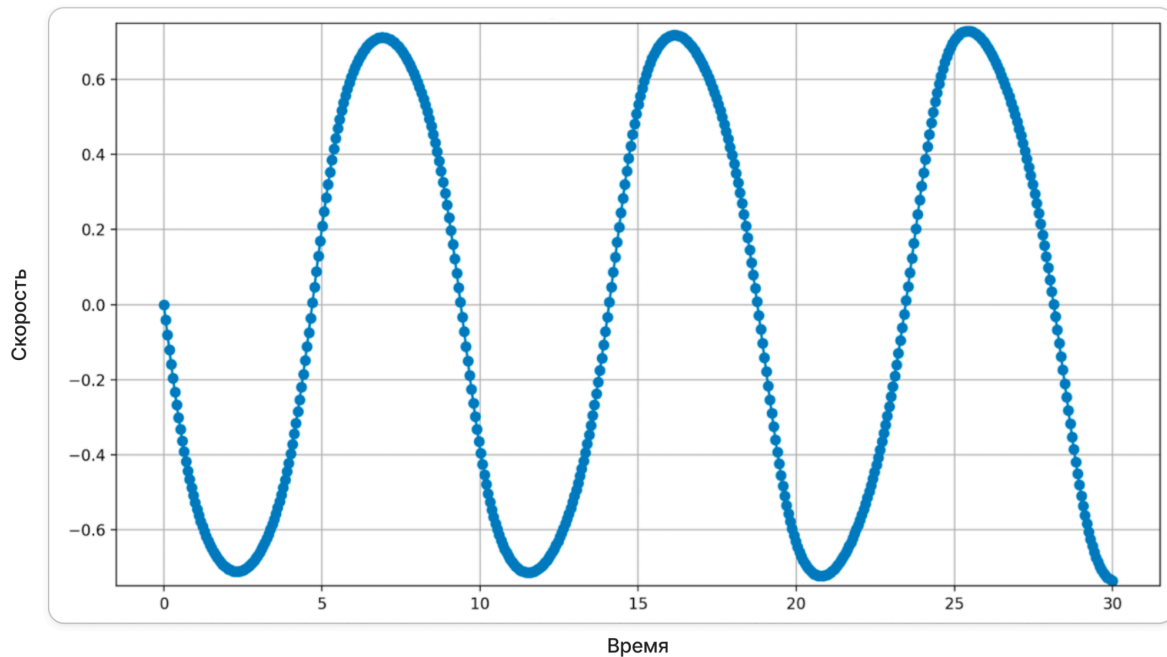


Рисунок 10. Временная зависимость для вертикальной скорости точки поверхности массива с необработанным файлом без динамической релаксации (ограничение по Y от -0.75 до 0.75)

В результате обработки файлов было выставлено начальное напряжение в подобласти. Полученная в результате расчета осадки с обработанным файлом временная зависимость для вертикальной скорости точки поверхности массива сплошной среды, показана на рисунках 11 и 12

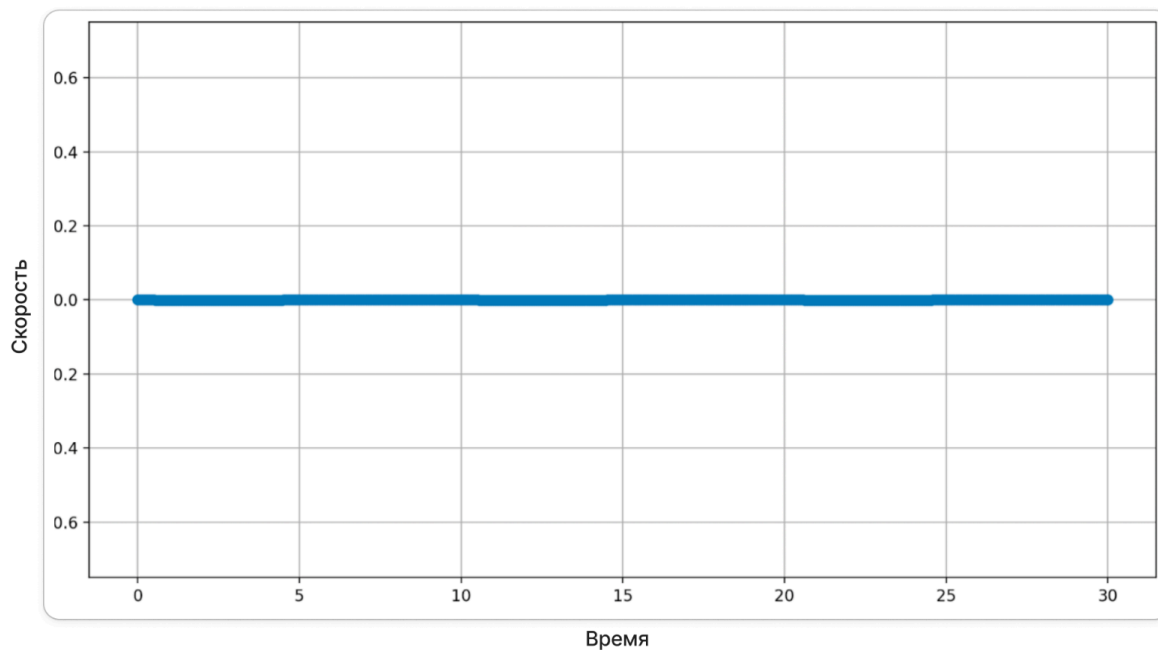


Рисунок 11. Временная зависимость для вертикальной скорости точки поверхности массива с обработанным файлом с динамической релаксацией (ограничение по Y от -0.75 до 0.75)

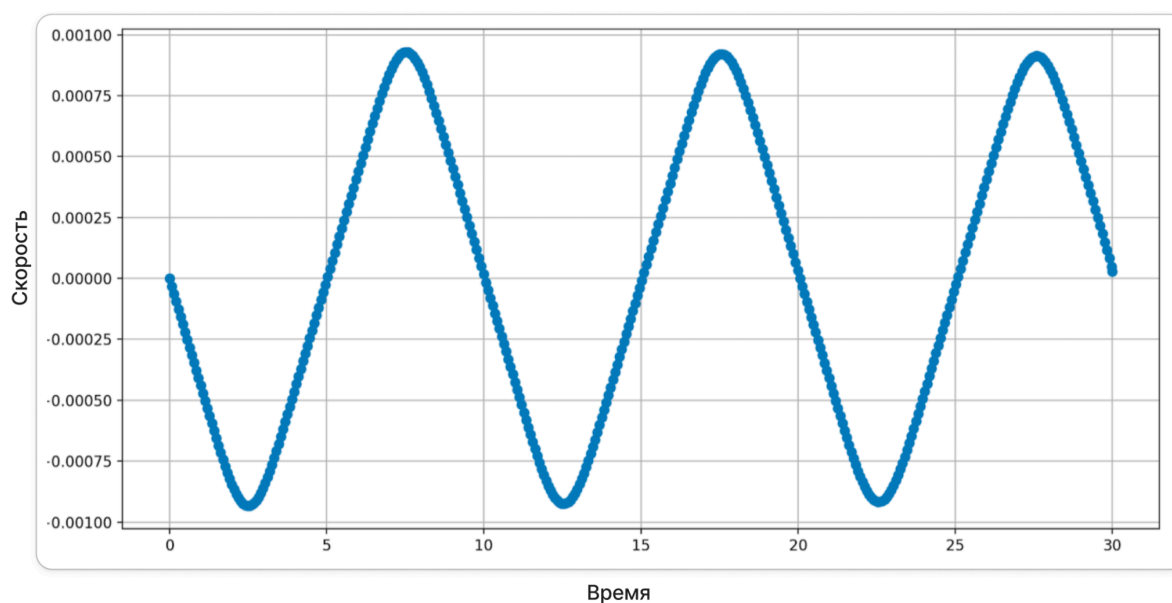


Рисунок 12. Временная зависимость для вертикальной скорости точки поверхности массива с необработанным файлом с динамической релаксацией (ограничение по Y от -0.001 до 0.001)

Видим, что задание начального поля позволило значительно уменьшить остаточные колебания массива сплошной среды от приложения поля силы тяжести. Таким образом, определение НДС подобласти от поля силы тяжести проведено с большей точностью.

Программа автоматизированного построения наборов элементов и задания начальных полей напряжений позволила упростить и ускорить этот процесс.

9. Тестирование

Методика тестирования

1. Тестирование с помощью скрипта (сравнение двух файлов)

Для тестирования использовались тестовые данные, которые были подсчитаны вручную, далее, была написана еще одна программа которая сравнивает строки двух файлов, и строки, которые не совпадают, записываются для дальнейшего поиска ошибок. Программа представляет собой небольшой скрипт в 41 строчку кода (рисунок 13):

```
# file1 - файл который будет сравниваться, в котором будет больше данных
def process_files(file1_path: str, file2_path: str, output_path: str) -> None: 1 usage
    """Обрабатывает файлы, удаляя блоки строк, если их заголовок найден в file2"""

    with open(file2_path, "r", encoding="utf-8", errors="ignore") as f2:
        lines_f2 = f2.readlines()

    result = []
    skip_next = False

    with open(file1_path, "r", encoding="utf-8", errors="ignore") as f1:
        lines_f1 = f1.readlines()
        i = 0

        while i < len(lines_f1):
            line = lines_f1[i].rstrip() # rstrip - убираем пробелы в конце строки

            skip_one_index = True

            if not line.startswith((" ", "\t")): # строка не начинается с пробела
                found = any(line_f2.strip() == line for line_f2 in lines_f2)

                if not found: # Если не нашли строку, добавляем её в результат
                    result.append(line)

                skip_next = found

            elif skip_next:
                skip_one_index = False
                while i < len(lines_f1) and lines_f1[i].startswith((" ", "\t")):
                    i += 1

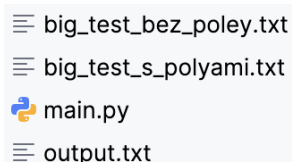
            else:
                result.append(line)

            if skip_one_index:
                i += 1

        with open(output_path, "w", encoding="utf-8") as output_file:
            for line in result:
                output_file.write(line + "\n")
```

Рисунок 13. Скрипт для тестирования

Входные файлы имеют название `_bez_poley.txt` и `_s_polyami.txt`. Первый файл, это файл, который нужно протестировать, а второй файл представляет собой требуемый файл с соблюдением всех правил написания для корректного запуска в “Логос Прочность” (рисунок 14)



```
big_test_bez_poley.txt
big_test_s_polyami.txt
main.py
output.txt
```

Рисунок 14. Входные файлы, файл скрипта и файл на выходе

2. Тестирование в программе “Логос Прочность”

Также, тестирование проводилось вручную. Файл, полученный программно, запускался в программе “Логос Прочность” и проводилось тестирование в следующей форме:

- Проверка того, что файл открывается корректно
- Проверялось, что все узлы остались на своем месте
- Проверялась правильность расчетов напряжения для каждого слоя

Результаты тестирования

В результате тестирования обоими способами обнаружено, что программа работает корректно на всех тестах, но при больших тестовых данных программа подвисала и не давала понять, будет ли продолжение работы, в дальнейшем это тоже было исправлено – добавлена информация о работе программы в текстовое поле, а также была добавлена асинхронная обработка с помощью потоков, чтобы программа не зависала.

Также, при работе с очень большими входными данными (порядка 1 млн элементов) расчет первоначально занимал 4–5 часов, при этом загрузка процессора не превышала 25–30%, а использование оперативной памяти оставалось практически постоянным. Это создавало впечатление зависания программы, хотя фактически она продолжала выполнение. В процессе анализа было выявлено, что основное время тратилось на неоптимальную обработку и хранение данных, включая многократные проходы по большим структурам и операции с высокой алгоритмической сложностью. После переработки методов хранения данных, сокращения количества вложенных циклов и устранения повторной обработки одних и тех же элементов время выполнения было значительно сокращено. В результате обработка того же файла с аналогичным объемом данных теперь занимает около 10 минут, что свидетельствует о существенном повышении эффективности реализованного решения.

10. Приложения

Ссылки:

- Документация PyInstaller (<https://pyinstaller.org/>)
- Документация PyYaml (<https://pyyaml.org/wiki/PyYAMLDocumentation>)
- Документация click (<https://click.palletsprojects.com/en/stable/>)
- Документация threading (<https://docs.python.org/3/library/threading.html>)
- Документация os (<https://docs.python.org/3/library/os.html>)
- Страничка работы на GitHub (<https://github.com/Kurillccc/ParserKFile>)
- Блок-схема в Figma (<https://clcl.li/knNme>)